Introduction to Theoretical Computer Science

CS2490 - Session 3: A first glance at automata

Pierre Cagne

Fall 2026

August 21, 2026

Example

Let Σ be an alphabet.

Example

Let Σ be an alphabet. What is the language Σ^*?

Example

Let Σ be an alphabet. What is the language Σ^*?

Let's see:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \cdots \cup \Sigma^n \cup \ldots$$

Example

Let Σ be an alphabet. What is the language Σ^*?



Let's see:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \cdots \cup \Sigma^n \cup \ldots$$

Example

Let Σ be an alphabet. What is the language Σ^*?

Let's see:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \cdots \cup \Sigma^n \cup \ldots$$

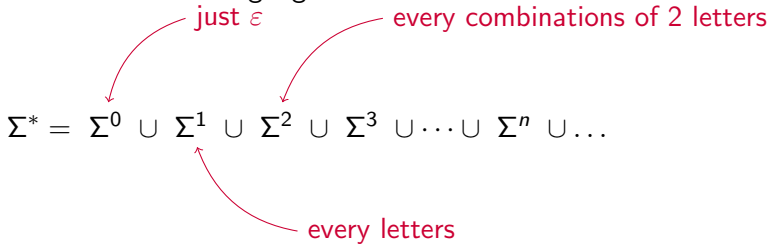
just ε

every letters

Example

Let Σ be an alphabet. What is the language Σ^*?

Let's see:



Example

Let Σ be an alphabet. What is the language Σ^*?

Let's see:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \cdots \cup \Sigma^n \cup \ldots$$

just ε

every combinations of 2 letters

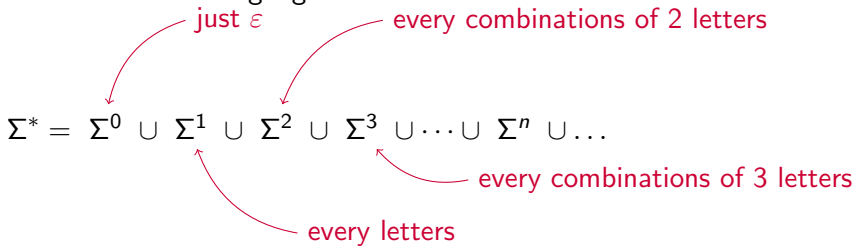
every combinations of 3 letters

every letters

Example

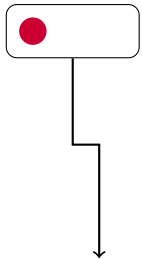
Let Σ be an alphabet. What is the language Σ^*?

Let's see:



Σ^* is the set of all words over Σ.

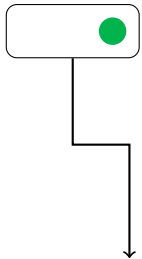
Recall from session 1



Rule: in state , reading 1
change to state

1101100111010110001

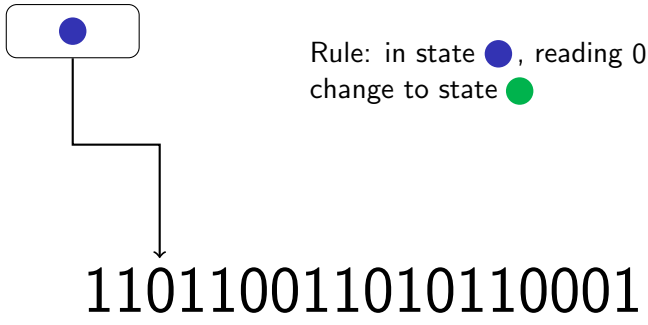
Recall from session 1



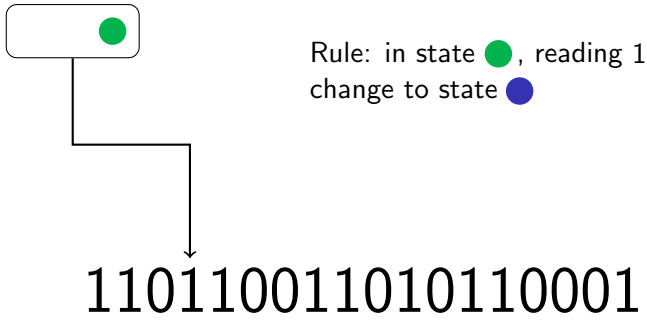
Rule: in state , reading 1
change to state

1101100111010110001

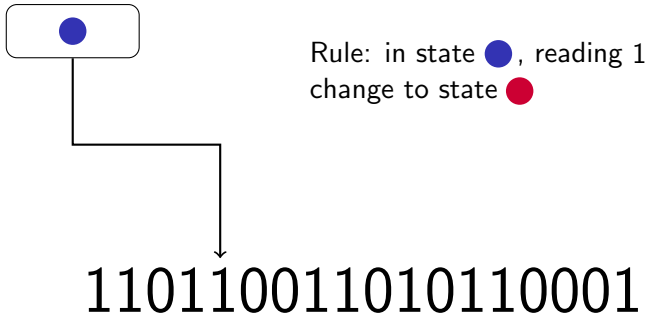
Recall from session 1



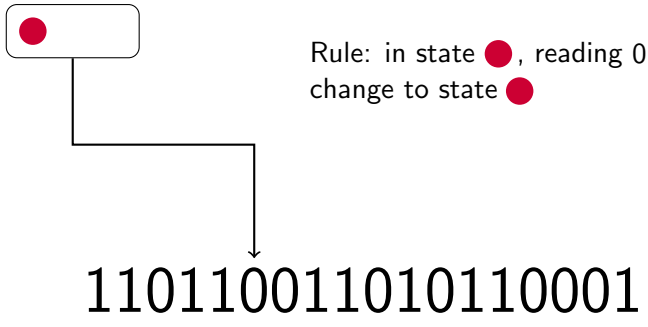
Recall from session 1



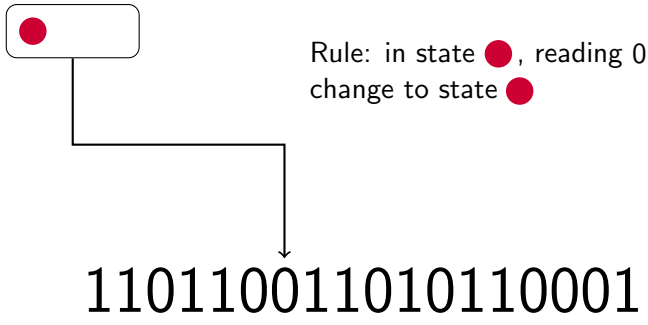
Recall from session 1



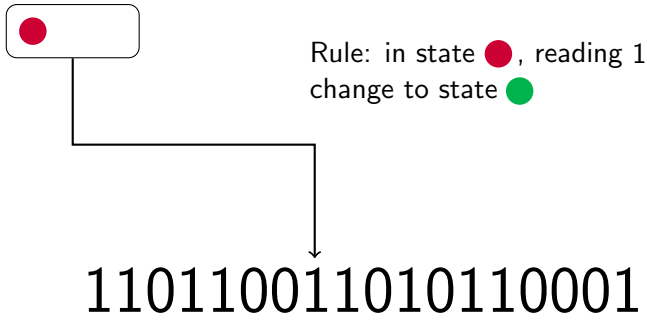
Recall from session 1



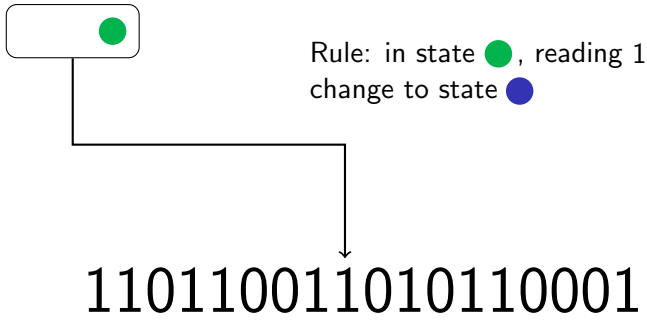
Recall from session 1



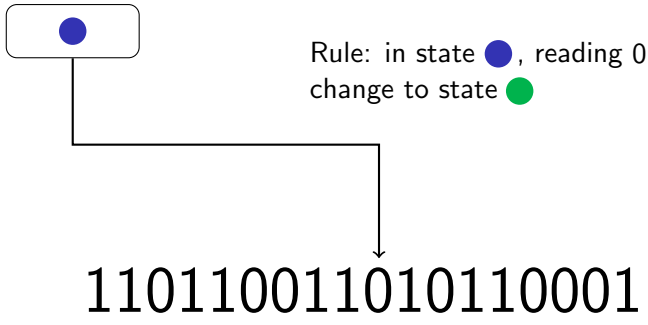
Recall from session 1



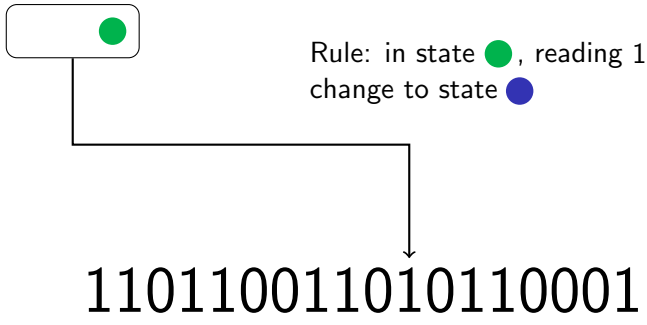
Recall from session 1



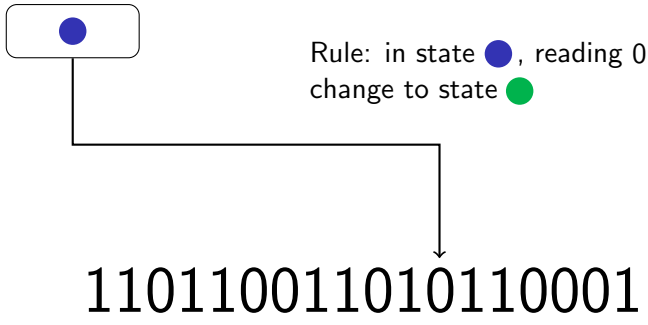
Recall from session 1



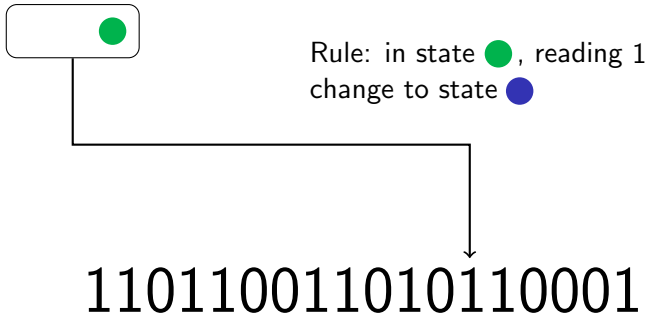
Recall from session 1



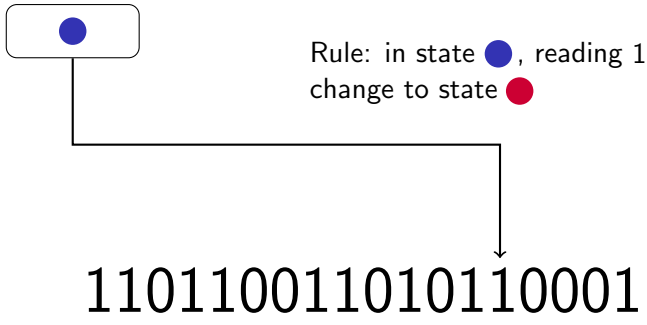
Recall from session 1



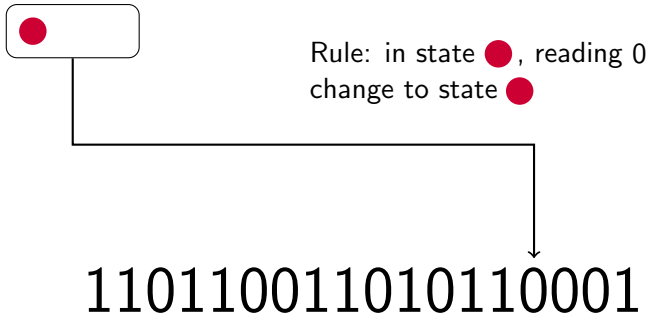
Recall from session 1



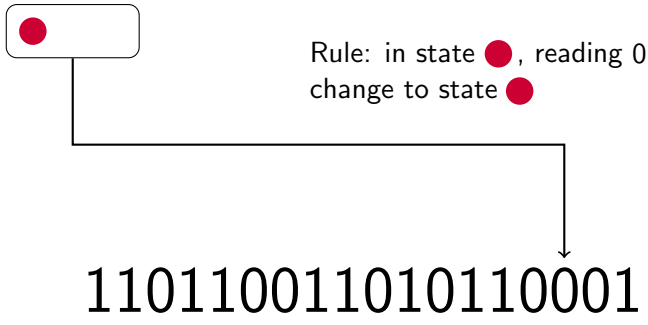
Recall from session 1



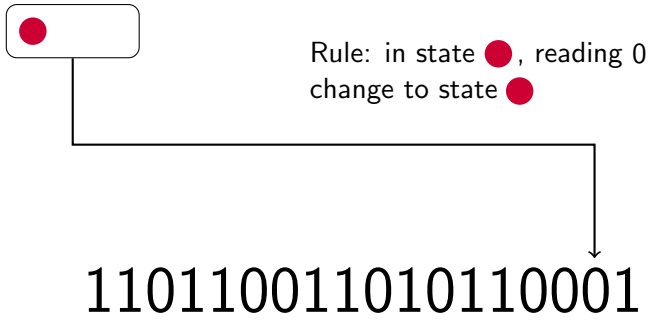
Recall from session 1



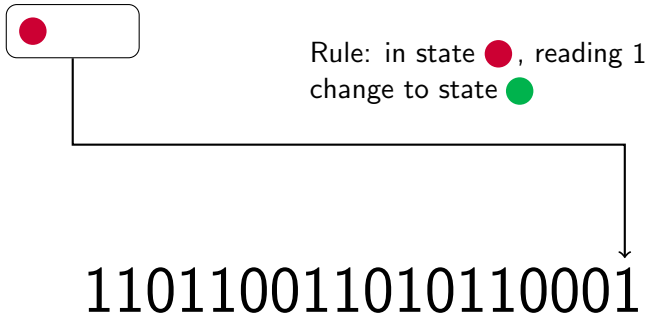
Recall from session 1



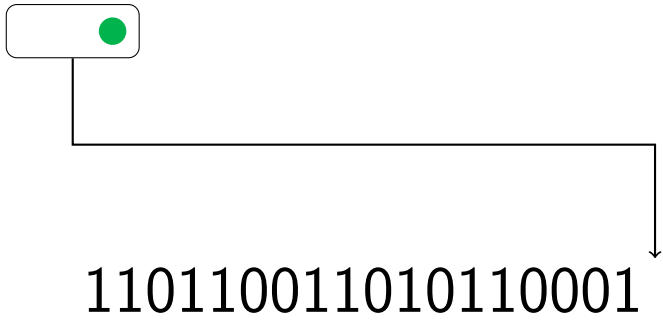
Recall from session 1



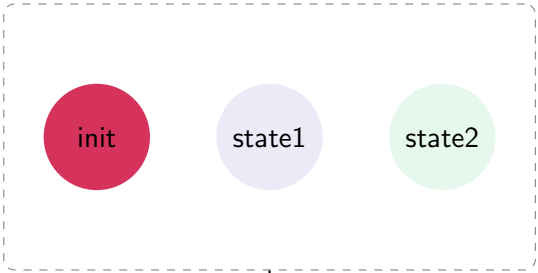
Recall from session 1



Recall from session 1



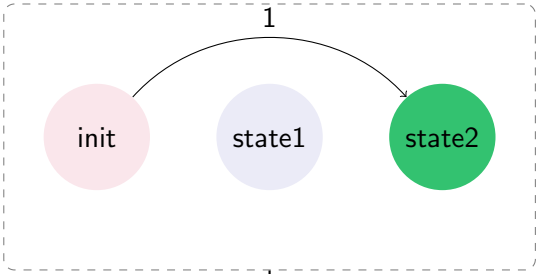
Let's open the hood



Rule: in state ●, reading 1
change to state ●

1101100110101100 01

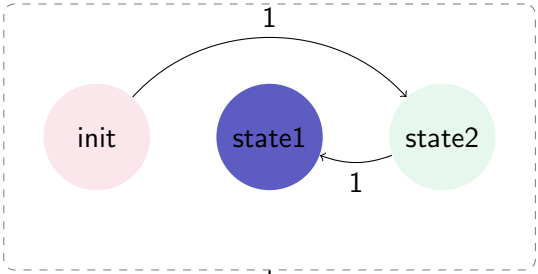
Let's open the hood



Rule: in state , reading 1
change to state

1101100110101101 0001

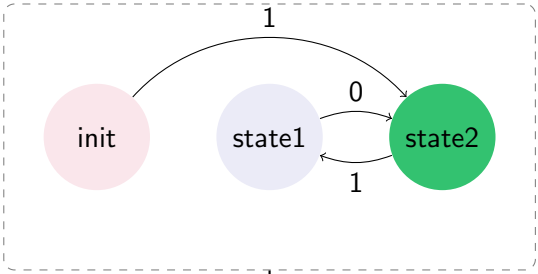
Let's open the hood



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

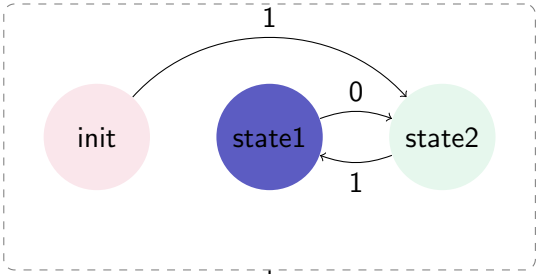
Let's open the hood



Rule: in state ●, reading 1
change to state ●

1101100110101101 0001

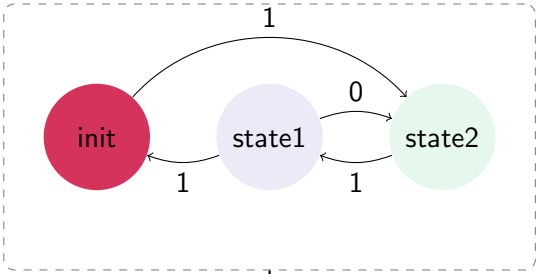
Let's open the hood



Rule: in state , reading 1
change to state

1101100110101101 0001

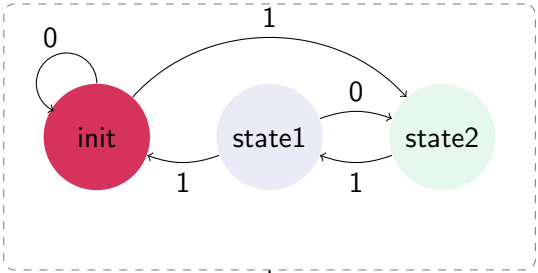
Let's open the hood



Rule: in state ●, reading 0
change to state ●

1101100110101101 0001

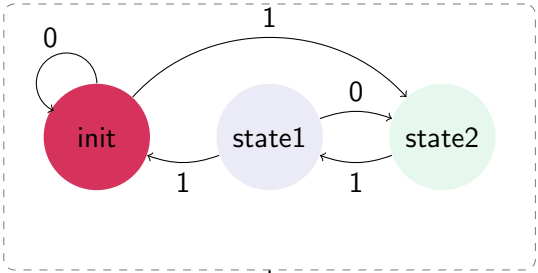
Let's open the hood



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

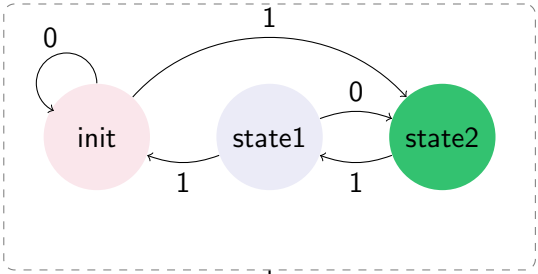
Let's open the hood



Rule: in state  , reading 1
change to state 

1101100110101100 01

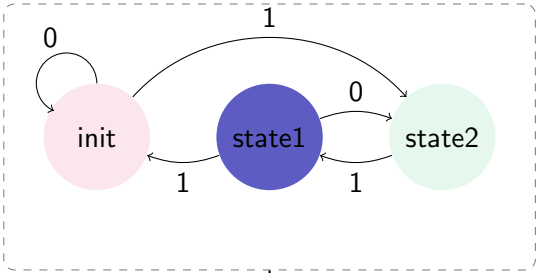
Let's open the hood



Rule: in state , reading 1
change to state

1101100110101100 01

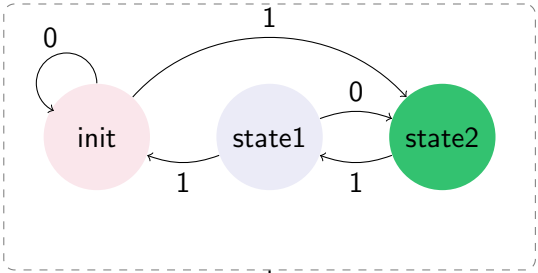
Let's open the hood



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

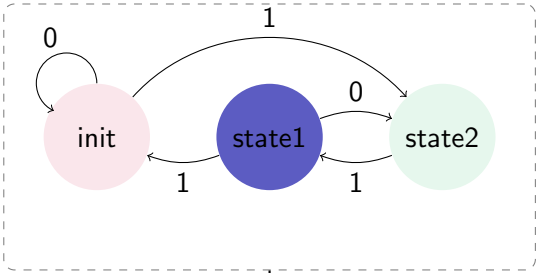
Let's open the hood



Rule: in state ●, reading 1
change to state ●

1101100110101100 01

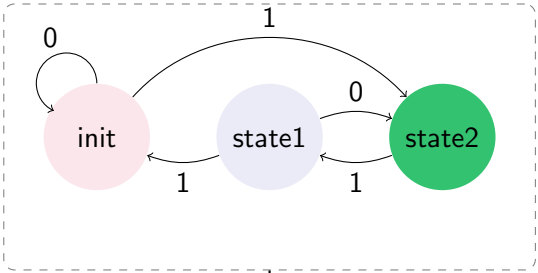
Let's open the hood



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

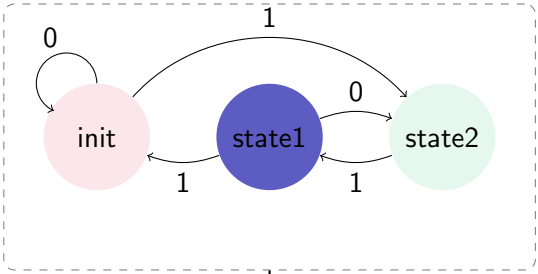
Let's open the hood



Rule: in state ●, reading 1
change to state ●

1101100110101100 01

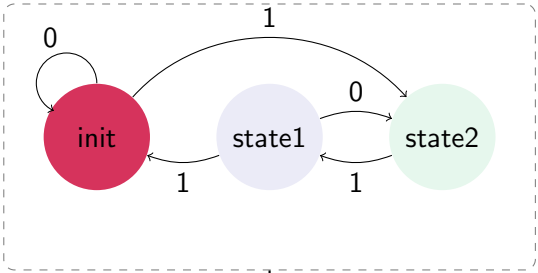
Let's open the hood



Rule: in state , reading 1
change to state

1101100110101100 01

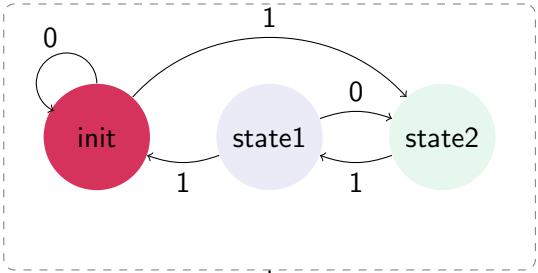
Let's open the hood



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

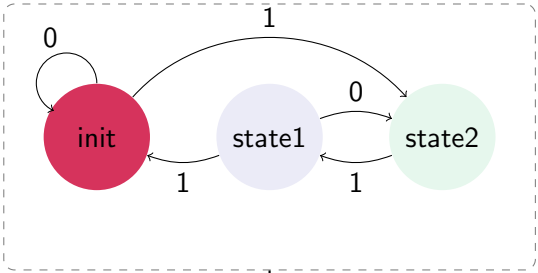
Let's open the hood



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

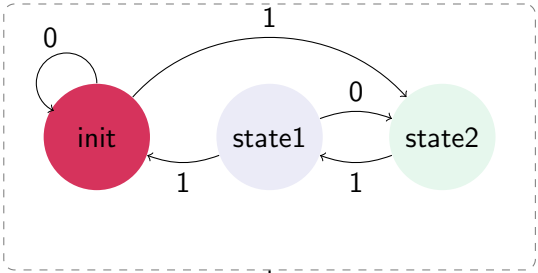
Let's open the hood



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

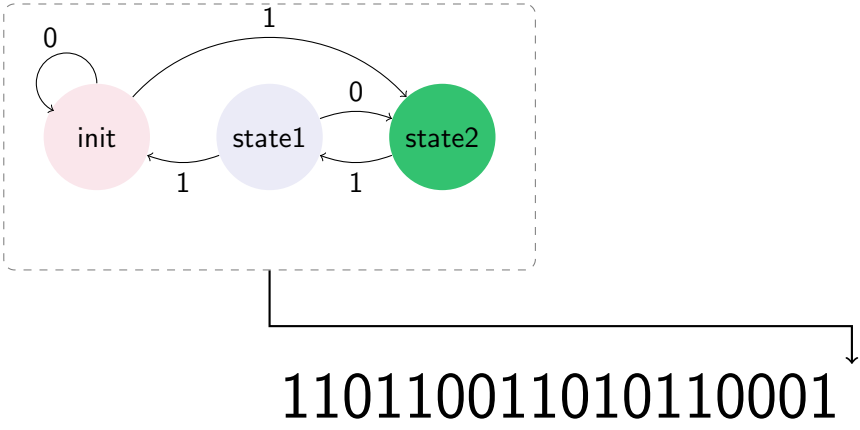
Let's open the hood



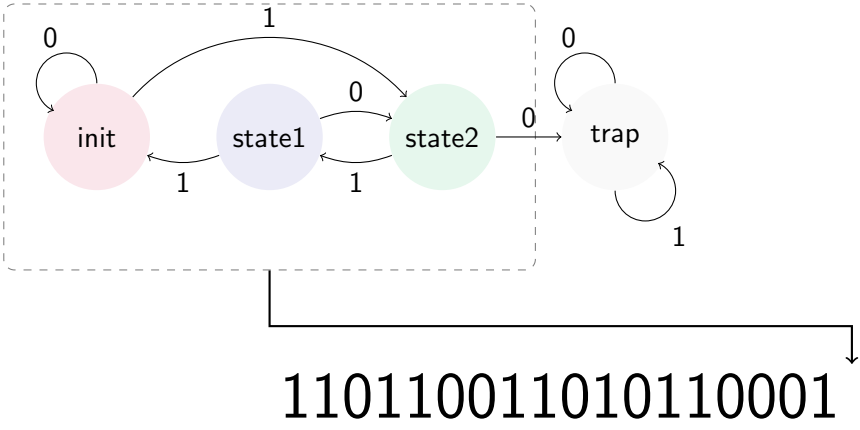
Rule: in state  , reading 1
change to state 

1101100110101100 01

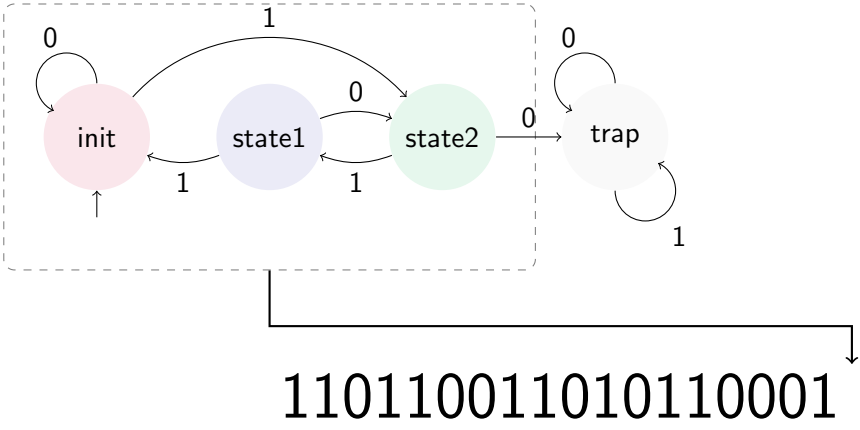
Let's open the hood



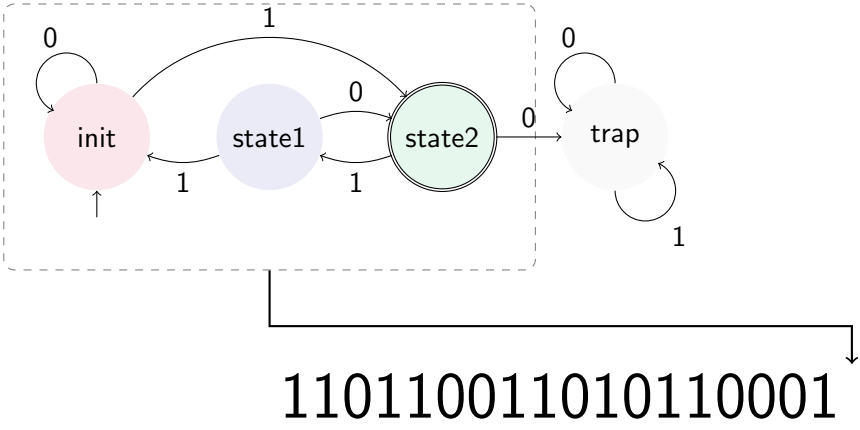
Let's open the hood



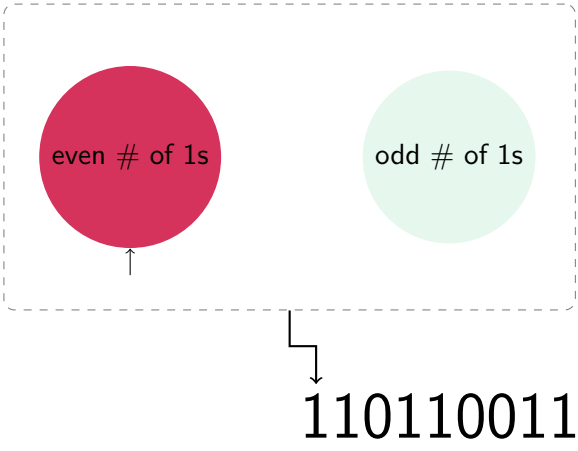
Let's open the hood



Let's open the hood

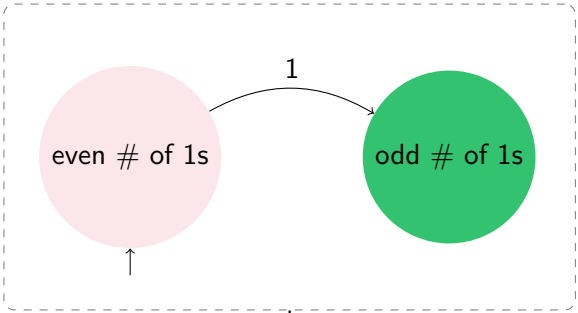


Let's run it again: odd number of 1s



Rule: in state  , reading 1
change to state 

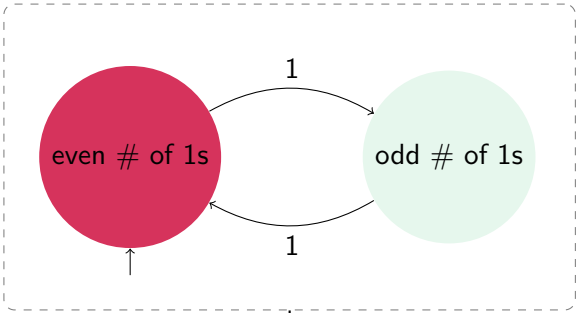
Let's run it again: odd number of 1s



Rule: in state  , reading 1
change to state 

1101100110101100 01

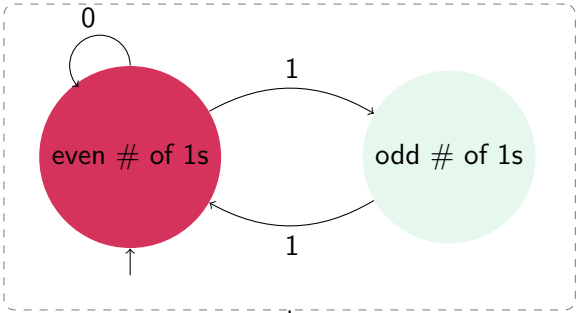
Let's run it again: odd number of 1s



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

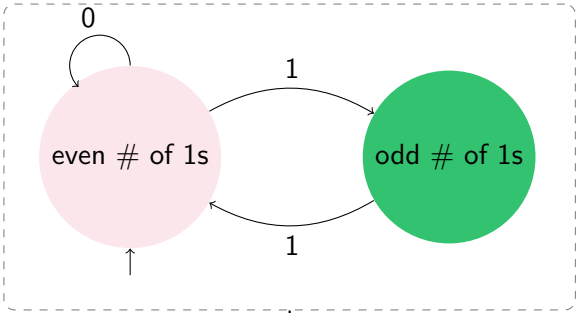
Let's run it again: odd number of 1s



Rule: in state  , reading 1
change to state 

1101100110101100 01

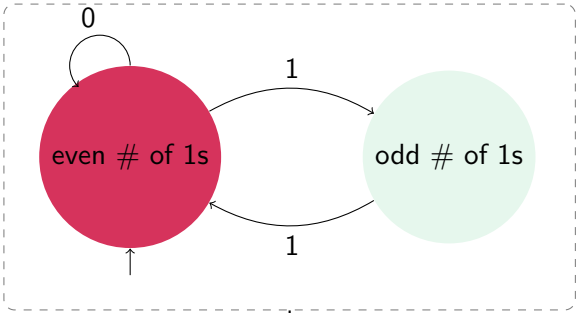
Let's run it again: odd number of 1s



Rule: in state  , reading 1
change to state 

1101100110101100 01

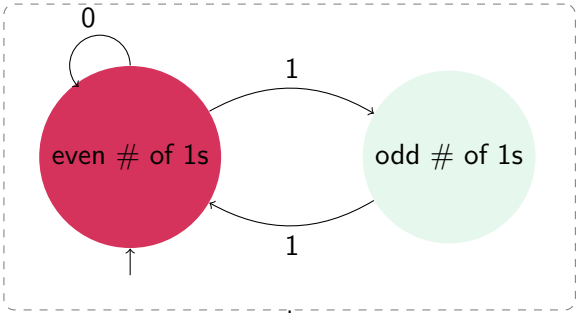
Let's run it again: odd number of 1s



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

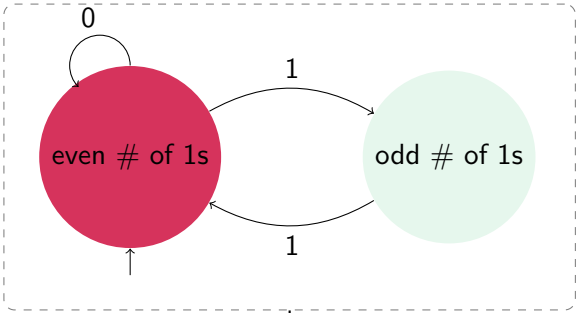
Let's run it again: odd number of 1s



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

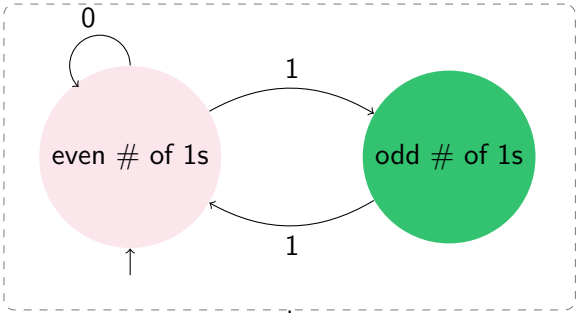
Let's run it again: odd number of 1s



Rule: in state  , reading 1
change to state 

1101100110101100 01

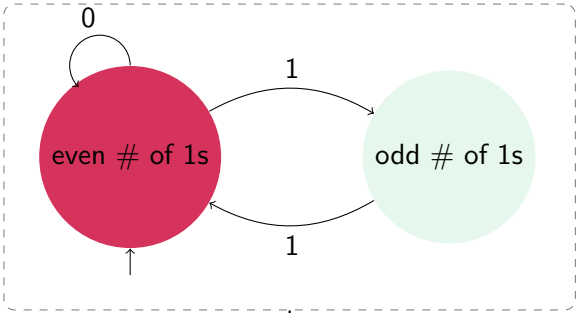
Let's run it again: odd number of 1s



Rule: in state  , reading 1
change to state 

1101100110101100 01

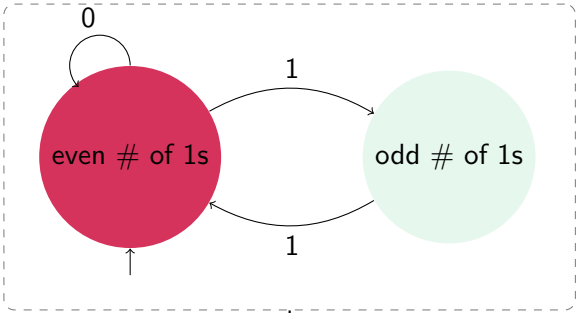
Let's run it again: odd number of 1s



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

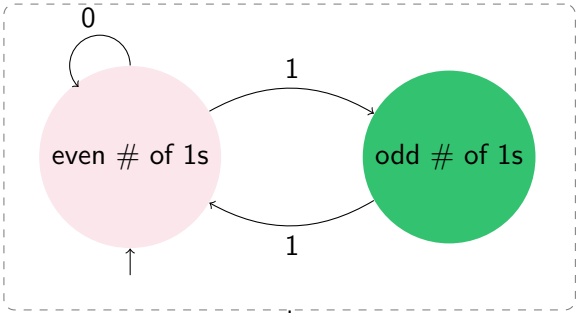
Let's run it again: odd number of 1s



Rule: in state  , reading 1
change to state 

1101100110101100 01

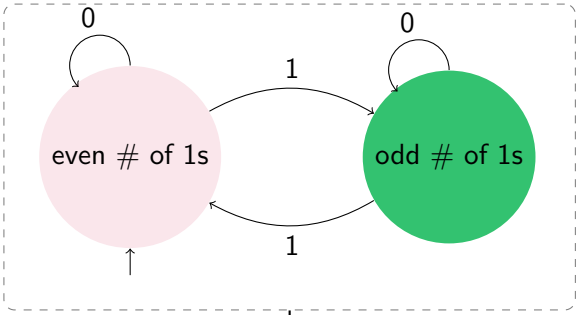
Let's run it again: odd number of 1s



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

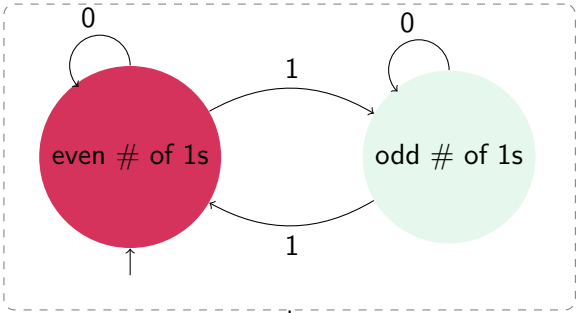
Let's run it again: odd number of 1s



Rule: in state ●, reading 1
change to state ●

1101100110101100 01

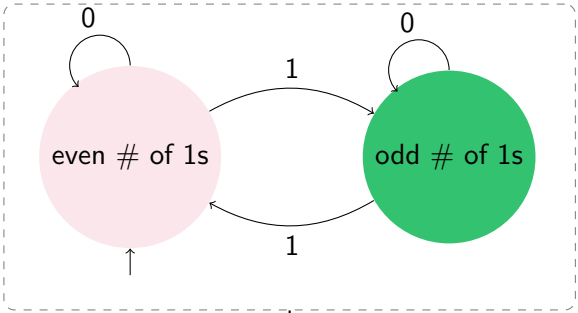
Let's run it again: odd number of 1s



Rule: in state ●, reading 1
change to state ●

1101100110101100 01

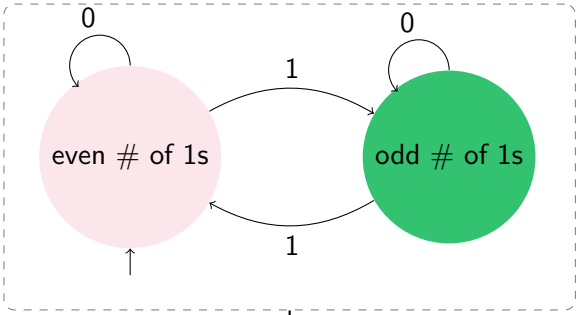
Let's run it again: odd number of 1s



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

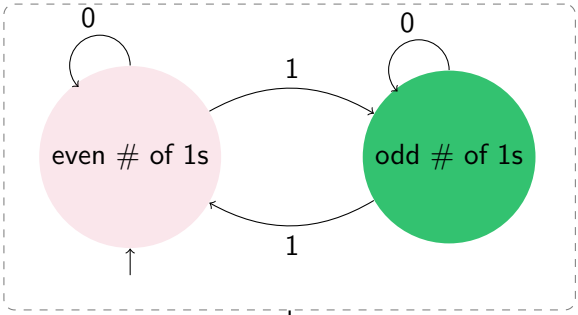
Let's run it again: odd number of 1s



Rule: in state ●, reading 0
change to state ●

1101100110101100 01

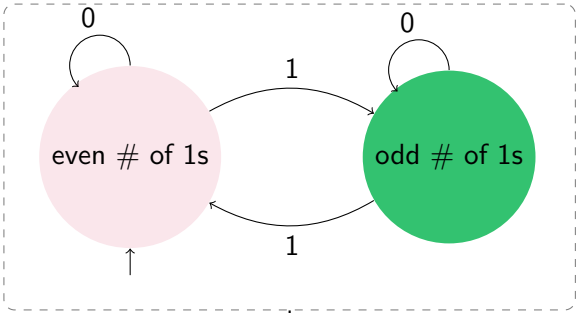
Let's run it again: odd number of 1s



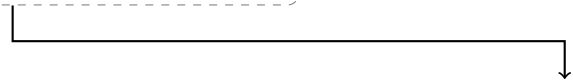
Rule: in state ●, reading 0
change to state ●

1101100110101100 01

Let's run it again: odd number of 1s

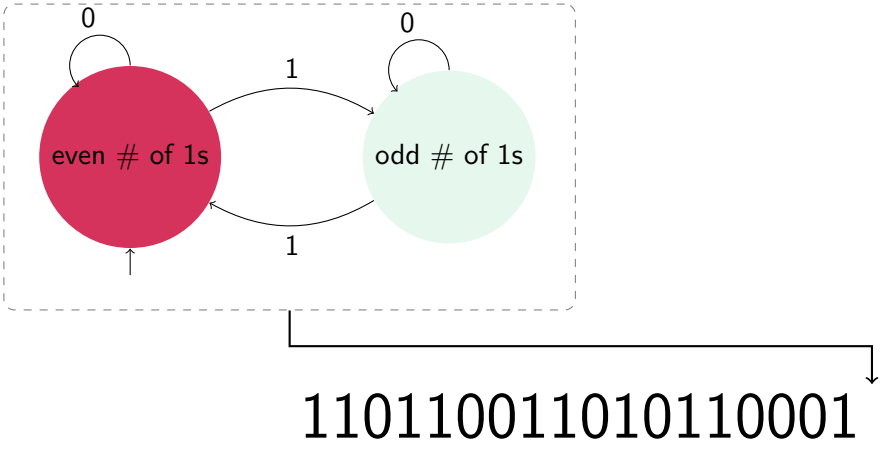


Rule: in state  , reading 1
change to state 

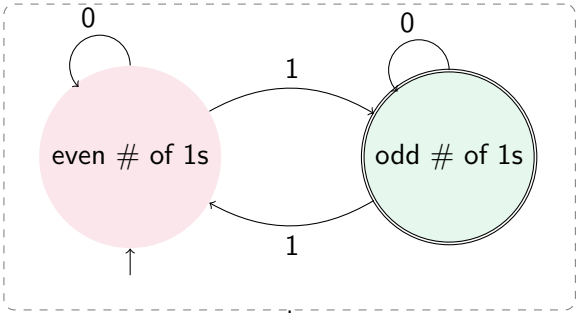


1101100110101100 01

Let's run it again: odd number of 1s



Let's run it again: odd number of 1s



1101100110101100 01

Definition of DFA

Let Σ be an alphabet. A deterministic finite automaton (DFA) $\mathcal{A}$ is given by:

Definition of DFA

Let Σ be an alphabet. A deterministic finite automaton (DFA) $\mathcal{A}$ is given by:

- a finite set Q: each element is a name of a state,

Definition of DFA

Let Σ be an alphabet. A deterministic finite automaton (DFA) $\mathcal{A}$ is given by:

- a finite set Q: each element is a name of a state,
- an initial state $q_0 \in Q$: the state in which all computations begin,

Definition of DFA

Let Σ be an alphabet. A deterministic finite automaton (DFA) $\mathcal{A}$ is given by:

- a finite set Q: each element is a name of a state,
- an initial state $q_0 \in Q$: the state in which all computations begin,
- a subset $F \subseteq Q$: each element is a name of an accepting/final state,

Definition of DFA

Let Σ be an alphabet. A deterministic finite automaton (DFA) $\mathcal{A}$ is given by:

- a finite set Q: each element is a name of a state,
- an initial state $q_0 \in Q$: the state in which all computations begin,
- a subset $F \subseteq Q$: each element is a name of an accepting/final state,
- a function $\delta : \mathbf{\Sigma} \times \mathbf{Q} \to \mathbf{Q}$ (called the transition function): when in state q, and reading a, change to state $\delta(a, q)$.

Definition of DFA

Let Σ be an alphabet. A deterministic finite automaton (DFA) $\mathcal{A}$ is given by:

- a finite set Q: each element is a name of a state,
- an initial state $q_0 \in Q$: the state in which all computations begin,
- a subset $F \subseteq Q$: each element is a name of an accepting/final state,
- a function $\delta : \mathbf{\Sigma} \times \mathbf{Q} \to \mathbf{Q}$ (called the transition function): when in state q, and reading a, change to state $\delta(a, q)$.

We write $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$.